

CSCE 670 INFORMATION RETRIEVAL · MODULE 07 · LECTURE NOTES

Dense Retrieval

Matching meaning, and when it helps.

MODULE OUTCOMES

- Explain the bi-encoder and cross-encoder designs.
- Explain how a bi-encoder is trained.
- Explain where dense retrieval wins and where it loses.
- Explain hybrid retrieval and why it dominates.
- Build and evaluate a dense retriever.

1 Two designs

	Bi-encoder	Cross-encoder
Input	Query and document encoded separately.	Query and document encoded together, as one sequence.
Score	Dot product or cosine of the two vectors.	A model output computed over the pair.
Precompute	All documents, offline, before any query arrives.	Nothing — it fundamentally needs the query.
Cost per query	One query encode, then a vector search (Module 08).	One full model pass per candidate document.
Quality	Good.	Better — the query and document terms interact (see the callout).
Use it as	First-stage retrieval.	A reranker (Module 06 §4).

The precompute row determines everything else in the table — a bi-encoder can index the entire collection in advance and a cross-encoder fundamentally cannot, since its computation depends on the query — which is why each design has its own stage in the standard architecture. The precompute argument explains the whole architecture, and it is worth being able to derive the two-stage design from it rather than learning it as a convention.

A CROSS-ENCODER IS BETTER BECAUSE THE QUERY AND DOCUMENT TERMS CAN INTERACT

A bi-encoder must compress an entire document into a single fixed vector *before* it knows what the query will be — so it has to anticipate every question anybody might ever ask of that document, which a fixed-size vector simply cannot do.

A cross-encoder sees both the query and the document together, so attention can match a specific query term against the specific part of the document that addresses it (CSCE 638 Module 06 §1's lookup framing, used for exactly this).

Which is a representational difference rather than a capacity one — a larger bi-encoder does not close the gap, because the bottleneck is the single vector rather than the model's expressiveness, which is §3's structural limit.

And it is the whole justification for the two-stage architecture — use the cheap design to narrow the candidate set and the good design to order it (Module 06 §4), which gets most of the cross-encoder's quality at a fraction of its cost.

2 Training

THE OBJECTIVE

contrastive: pull a query and its relevant document together in the embedding space, and push the query and irrelevant documents apart.

THE NEGATIVES ARE EVERYTHING

random negatives	easy, and the model learns only coarse topical similarity from them
in-batch negatives	free -- use the other queries' documents in the same batch. Bigger batch, more negatives, better model.
hard negatives	documents retrieved by BM25 and judged not relevant. The ones that teach the actual distinction.
mined from the model	itself, iteratively, which is better again and risks false negatives

AND FALSE NEGATIVES ARE THE TRAP: an unlabelled but genuinely relevant document used as a negative teaches the model exactly the wrong thing.

Hard negative mining is where the quality comes from — the architecture is largely settled and the negatives, not the architecture, determine dense retrieval quality, which is a finding worth knowing before spending time on model variants. And false negatives are its characteristic failure: because

relevance judgements are incomplete (Module 05 §1's pooling), the hardest mined negatives are disproportionately likely to be unlabelled relevant documents, which actively damages the model.

3 Where it wins and loses

- **Dense retrieval wins on paraphrase and vocabulary mismatch** — which is **Module 01 §2's central problem**, and **this is the first technique in the course that attacks it directly** rather than by term substitution.
- **And on short queries**, where **there are too few terms for lexical matching to work with** — a two-word query gives BM25 very little to compute with and gives an encoder enough to represent an intent.
- **Lexical retrieval wins on exact terms: identifiers, product codes, personal names, numbers, version strings, and rare technical vocabulary** — all of which **a fixed-dimension vector blurs together with its neighbours**, because that is what the embedding is for.
- **And on out-of-domain collections**, where **the dense model was trained on something else and BM25 requires no training at all** — **which the zero-shot benchmarks showed clearly** and which was not the expectation.
- **So the failure modes are complementary rather than overlapping**, which is §4's entire argument. **The out-of-domain result is the one that corrected the field's expectations** — dense retrievers transfer considerably less well than their in-domain numbers had suggested, and BM25's training-free generality turned out to be worth more than it looked.

AND THE SINGLE-VECTOR BOTTLENECK IS THE STRUCTURAL LIMIT

A document of two thousand words compressed into 768 numbers has lost the great majority of itself — and critically, **which parts it kept were decided before the query was known** (§1's callout).

So a document covering several distinct topics is represented as something like an average of them, matching none of them well — which is exactly **why chunking matters so much in practice** (CSCE 638 Module 11 §3's retrieve-small-generate-large pattern).

The multi-vector designs address this directly by storing a vector per token and matching query tokens against document tokens individually — **much better quality, and a very much larger index**, which is the cost.

Which is the governing trade in this area: more vectors per document buys quality and costs storage and search time, and the right point on that curve depends entirely on your constraints rather than on which is better in the abstract.

4 Hybrid retrieval

COMBINE LEXICAL AND DENSE RETRIEVAL, BECAUSE THEIR FAILURES ARE UNCORRELATED

Run both and fuse the rankings — reciprocal rank fusion is the standard method, needs no tuning, and requires no score calibration between the two systems, which makes it the right first attempt and frequently the last one.

And the gain is real and consistent across collections, because **the two methods fail on different queries** (§3) rather than on the same ones — which is the condition under which combining two systems helps at all.

With a learned combination doing better still if you have relevance judgements — **the two scores simply become features for Module 06's ranker**, alongside everything else, which is how production systems actually do it.

So the production shape is: hybrid first-stage retrieval, then a cross-encoder reranker — which is where this course's Modules 03, 06, and 07 converge, and is the current standard architecture for a serious search system.

- **Recall at your first-stage cutoff**, measured for each method separately and for the fusion — **which shows whether the hybrid is actually adding anything** or whether one method is doing all the work.
- **Performance on in-domain and out-of-domain queries separately**, because **the gap is large** (§3) and an aggregate figure over a mixed query set tells you nothing useful about either.
- **Performance on exact-match queries specifically** — identifiers, product codes, personal names — which is **where a dense-only system fails embarrassingly** and where users notice immediately.
- **The index size and the query latency**, since **both are substantially larger than for a lexical index** (Module 08) and both are real operating costs.
- **And the embedding model's version**, because **changing it invalidates the entire index** — every document must be re-encoded — **and that is an operational fact worth planning for** rather than discovering. **Changing the embedding model means reindexing everything, which makes the model choice a commitment rather than a configuration.**

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
Lin, Nogueira & Yates — Pretrained Transformers for Text Ranking (free)	The whole module — and the most careful available assessment of which results replicated.
Karpukhin et al. — Dense Passage Retrieval (free)	§2's training in the original — including the in-batch negatives finding.
Thakur et al. — BEIR (free)	§3's out-of-domain result — the benchmark that showed how much less dense retrieval transfers.

Resource	What it covers for this module
Khattab & Zaharia — ColBERT (free)	§3's multi-vector answer — late interaction, and the index size it costs.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 **Build a bi-encoder retriever** over your collection and measure recall at 100.
- 2 **Add a cross-encoder reranker** and measure the NDCG improvement.
- 3 **Measure the cross-encoder's latency per candidate** and compute the maximum candidates your budget allows.
- 4 **Train with random negatives, then in-batch, then BM25 hard negatives**, and compare.
- 5 **Find a false negative** among your mined hard negatives.
- 6 **Compare dense and BM25 on paraphrase queries** and on identifier queries.
- 7 **Evaluate your dense retriever on an out-of-domain collection** and report the gap.
- 8 **Fuse the two rankings with reciprocal rank fusion** and measure the gain.
- 9 **Measure recall for each method and the fusion** separately.
- 10 **Time a full reindex** with a different embedding model.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 Contrast bi-encoder and cross-encoder on six dimensions.
- 2 Which row determines the others, and why?
- 3 Why is a cross-encoder better, and why does a bigger bi-encoder not close the gap?
- 4 Describe the contrastive objective and the four kinds of negative.
- 5 Why are the negatives the whole problem, and what is the trap?
- 6 Where does dense retrieval win, and where does lexical?
- 7 What was the out-of-domain finding?
- 8 State the single-vector bottleneck and the multi-vector trade.
- 9 Why does hybrid retrieval work, and what is the standard fusion method?
- 10 Why is the embedding model choice a commitment?